# Project 1: Raytracing a Black Hole Image

PHYS 4027/5027

Due Date: 11:59pm Wednesday 10/22/2025

Have you seen the movie *Interstellar*? The ring of light surrounding the black hole in that movie is done realistically using a raytracing algorithm:



In this project, you will use a numerical ODE integrator to solve the geodesic equation for photons near the event horizon of a rotating black hole. Then you will use this tool to trace light rays near the black hole to create an image similar to the one above, which is the image of the accretion disk distorted by the spacetime of the black hole.

## 1 A Crash Course on General Relativity

We will use results from General Relativity (GR) to write down the equations for the trajectories of photons. It's okay if you don't know GR, since this section is written to provide some background

on the topic. We will introduce some concepts from GR, allowing us to write down the equations governing the photon motion in a curved spacetime.

## 1.1 The Kerr Spacetime

The spacetime is described by something called a "metric". The metric of a rotating black hole can be written as follows:

$$ds^2 = g_{tt}dt^2 + 2g_{t\phi}dtd\phi + g_{rr}dr^2 + g_{\theta\theta}d\theta^2 + g_{\phi\phi}d\phi^2. \tag{1}$$

In the 4-dimensional spacetime, the metric gives us a way to calculate the distance $ds$ between two points separated by some infinitesimal displacement $(dt, dr, d\theta, d\phi)$. You can think of this as a generalization of the Euclidean distance $ds^2 = dx^2 + dy^2 + dz^2$. Since the metric coefficients may be position-dependent, the total distance between two points need to be calculated using an integral.

For a spinning Kerr black hole, the metric functions look like this:

$$g_{tt} = \frac{2Mr}{\rho^2} - 1, \quad g_{t\phi} = -\frac{2Mar}{\rho^2}\sin^2\theta \tag{2}$$

$$g_{rr} = \frac{\rho^2}{\Delta}, \quad g_{\theta\theta} = \rho^2, \quad g_{\phi\phi} = \frac{\Sigma}{\rho^2}\sin^2\theta, \tag{3}$$

where we defined:

$$\rho^2 = r^2 + a^2\cos^2\theta, \tag{4}$$

$$\Delta = r^2 + a^2 - 2Mr, \tag{5}$$

$$\Sigma = (r^2 + a^2)^2 - a^2\Delta\sin^2\theta. \tag{6}$$

$M$ is the mass of the black hole, and $a$ is the spin parameter of the black hole. These quantities are written in units where speed of light $c = 1$ and the gravitational constant $G = 1$, so both $r$ and $a$ have the same unit as the mass $M$. Restoring $G$ and $c$, the unit of length should really be $GM/c^2$, which we call the *gravitational radius* $r_g$. In practice we often set $M = 1$ as well, which makes both $a$ and $r$ dimensionless. The dimensionless value of $a$ varies from $0$ to $1$, where $0$ corresponds to the black hole not spinning (the metric then reduces to a Schwarzschild black hole) and $1$ corresponds to the maximally spinning black hole (it can't spin faster than this).

The metric functions defined above are called the *Boyer-Lindquist* coordinates. There are different coordinate systems you can choose around a black hole, just like you can work with Cartesian or cylindrical or spherical coordinates in flat space. The Boyer-Lindquist coordinates are what we will adopt in this project. These coordinates have a singularity where $\Delta = 0$. The singularity is located at:

$$r_H^{\pm} = M \pm \sqrt{M^2 - a^2}. \tag{7}$$

The $+$ version gives the radius of the *event horizon* of the black hole, and we simply denote it $r_H$. This is the surface below which light can no longer escape, and is often quoted to be the radius of the black hole.

Before we go any further, we shall introduce the *Einstein summation notation*. It's a short hand to write the sum over several components:

$$u_\mu u^\mu = \sum_{\mu=0}^{3} u_\mu u^\mu = u_t u^t + u_r u^r + u_\theta u^\theta + u_\phi u^\phi. \tag{8}$$

As long as the same index is repeated both as a lower as well as an upper index, it is understood that it's summed over. The convention is that Greek indices like $\mu$ goes from 0 to 3, so they go over $(t, r, \theta, \phi)$, while alphabetical indices like $i$ goes from 1 to 3, only going over the spatial components $(r, \theta, \phi)$. In GR, upper indices and lower indices mean different things. If you are familiar with Quantum Mechanics, you can think of upper index vectors like $u^\mu$ analogous to ket vectors $|u\rangle$, while lower index vectors $u_\mu$ are like bra vectors $\langle u|$. Writing $u_\mu u^\mu$ is equivalent to taking their inner product $\langle u|u\rangle$.

## 1.2 The 3+1 Formalism

While standard textbooks will go from the metric we defined above and write down the *geodesic equation*, we will take a different approach. The standard geodesic equations are second order equations in time, which are inconvenient to integrate numerically. Instead, we will setup the problem following MacDonald & Thorne (1982) and Komissarov (2004). We define the $3 + 1$ split of the 4 dimensional spacetime into "absolute space" and time, writing the same metric (1) as the following:

$$ds^2 = (\beta^2 - \alpha^2)dt^2 + 2\beta_i dx^i dt + \gamma_{ij} dx^i dx^j. \tag{9}$$

Note that we have made use of the Einstein summation notation, and $x^i$ runs over $r, \theta, \phi$. The function $\alpha$ is called the *lapse function*, $\beta_i$ is called the *shift vector*, and $\gamma_{ij}$ is a metric for the absolute space. Note that at this stage, this is simply a rewrite of the metric (1). If we identify terms, we can write down the form of the lapse function, shift vector, and the spatial metric:

$$\alpha = \sqrt{\frac{\rho^2 \Delta}{\Sigma}}, \quad \beta_r = \beta_\theta = 0, \quad \beta_\phi = g_{t\phi}, \quad \gamma_{ij} = g_{ij}, \tag{10}$$

It is also important to define the *inverse metric* $\gamma^{ij}$, which satisfies the matrix equation $\gamma_{ij}\gamma^{jk} = \delta_{ik}$ (remember $j$ is summed over). In this case, since the spatial part of $g$ is diagonal, $\gamma_{ij}$ is diagonal, the inverse metric $\gamma^{ij}$ is simply $\gamma^{ij} = 1/g_{ij}$:

$$\gamma^{rr} = \frac{1}{g_{rr}} = \frac{\Delta}{\rho^2}, \quad \gamma^{\theta\theta} = \frac{1}{g_{\theta\theta}} = \frac{1}{\rho^2}, \quad \gamma^{\phi\phi} = \frac{1}{g_{\phi\phi}} = \frac{\rho^2}{\Sigma \sin^2\theta}. \tag{11}$$

All other components of $\gamma^{ij}$ are zero. The inverse metric can be used to define the upper-index version of the shift vector $\beta$: $\beta^i = \gamma^{ij}\beta_j$. Since the only nonzero component of $\beta_i$ is the $\phi$ component, and $\gamma^{ij}$ is diagonal, only the $\phi$ component of $\beta^i$ is nonzero:

$$\beta^r = \beta^\theta = 0, \quad \beta^\phi = \gamma^{\phi\phi}\beta_\phi = \frac{g_{t\phi}}{g_{\phi\phi}} = -\frac{2Mar}{\Sigma} \tag{12}$$

The full 4-dimensional spacetime metric can be expressed in terms of $\alpha$, $\beta_i$, and $\gamma_{ij}$:

$$g_{tt} = \beta^2 - \alpha^2, \quad g_{ti} = \beta_i, \quad g_{ij} = \gamma_{ij}. \tag{13}$$

Similarly, the inverse metric $g^{ij}$ can also be written in terms of $\alpha$, $\beta^i$, and $\gamma^{ij}$:

$$g^{tt} = -\frac{1}{\alpha^2}, \quad g^{ti} = \frac{\beta^i}{\alpha^2}, \quad g^{ij} = \gamma^{ij} - \frac{\beta^i \beta^j}{\alpha^2}. \tag{14}$$

In particular, since only the $\phi$ component of $\beta^i$ is nonzero, the only nonzero components of $g^{\mu\nu}$ are:

$$g^{tt} = -\frac{1}{\alpha^2}, \quad g^{t\phi} = \frac{\beta^\phi}{\alpha^2}, \quad g^{rr} = \gamma^{rr}, \quad g^{\theta\theta} = \gamma^{\theta\theta}, \quad g^{\phi\phi} = \gamma^{\phi\phi} - \left(\frac{\beta^\phi}{\alpha}\right)^2. \tag{15}$$

## 1.3 Equations of Motion for Photons

In the 3+1 formalism, the *Hamiltonian* for a massless particle like the photon is defined as follows:

$$H(x^i, u_i) = \alpha\sqrt{\gamma^{ij} u_i u_j} - u_i \beta^i, \tag{16}$$

where $x^i$ are the coordinates of the photon $r, \theta, \phi$; and $u_i$ are the momenta in these directions. The Hamiltonian defined with equation (16) coincides with the energy of the photon (just as a Hamiltonian should).[1]

The equations of motion for the photon can be now written down as the classical Hamilton's equations:

$$\frac{dx^i}{dt} = \frac{\partial H}{\partial u_i} = \gamma^{ij} u_i / u^t - \beta^i, \tag{17}$$

$$\frac{du_i}{dt} = -\frac{\partial H}{\partial x^i} = -\alpha u^t \partial_i \alpha + u_k \partial_i \beta^k - \frac{u_j u_k}{2u^t} \partial_i \gamma^{jk}, \tag{18}$$

where $\partial_i \alpha = \partial\alpha / \partial x^i$, and $u^t = \sqrt{\gamma^{ij} u_i u_j}/\alpha$. *Beware that $u^t$ and $u_t$ are different quantities!* The lower $u_t$ is negative of the energy of the photon and is expected to be a constant of motion:

$$u_t = -\alpha^2 u^t + u_i \beta^i = -H. \tag{19}$$

[1]Technically, energy is an observer-dependent quantity in GR, so here we mean the energy of the photon as viewed by a distant observer.

The equations can be expanded out in component form:

$$\frac{dr}{dt} = \gamma^{rr}\frac{u_r}{u^t} \tag{20}$$
$$\frac{d\theta}{dt} = \gamma^{\theta\theta}\frac{u_\theta}{u^t} \tag{21}$$
$$\frac{d\phi}{dt} = \gamma^{\phi\phi}\frac{u_\phi}{u^t} - \beta^\phi \tag{22}$$
$$\frac{du_r}{dt} = -\alpha u^t \partial_r \alpha + u_\phi \partial_r \beta^\phi - \frac{1}{2u^t}\left(u_r^2 \partial_r \gamma^{rr} + u_\theta^2 \partial_r \gamma^{\theta\theta} + u_\phi^2 \partial_r \gamma^{\phi\phi}\right) \tag{23}$$
$$\frac{du_\theta}{dt} = -\alpha u^t \partial_\theta \alpha + u_\phi \partial_\theta \beta^\phi - \frac{1}{2u^t}\left(u_r^2 \partial_\theta \gamma^{rr} + u_\theta^2 \partial_\theta \gamma^{\theta\theta} + u_\phi^2 \partial_\theta \gamma^{\phi\phi}\right) \tag{24}$$
$$\frac{du_\phi}{dt} = 0. \tag{25}$$

The final equation is simply the conservation of angular momentum, since $u_\phi$ can be identified with $L_z$, the angular momentum of the photon around the $z$ axis. Given a set of initial conditions $(r, \theta, \phi, u_r, u_\theta, u_\phi)$, these are the equations we will be solving in this project.

## 2 Implementation and Code Validation

Your first objective is to simply implement the equations (20)–(25) in C++, so that they can be plugged into an ODE solver that we have been discussing in class. Since you almost always evaluate the right hand sides of these equations at the same time, it's probably a good idea to write a simple class to do this:

```
class boyer_lindquist_metric {
 public:
  boyer_lindquist_metric(double a0, double M0) {
    // Initialize the parameters a and M
  }

  void compute_metric(double r, double th) {
    // Compute all the required metric components in one go
  }

  double a, M;
  double alpha, beta3;
  double gamma11, gamma22, gamma33; // components of upper gamma^ij
  double g_00, g_03, g_11, g_22, g_33; // components of lower g_\mu\nu
  double d_alpha_dr, d_beta3_dr, d_gamma11_dr, d_gamma22_dr, d_gamma33_dr;
  double d_alpha_dth, d_beta3_dth, d_gamma11_dth, d_gamma22_dth, d_gamma33_dth;
};
```

The reason that you might want to do this instead of writing an individual function for each component is so that you can reuse some results such as $\sin\theta$, $\cos\theta$, $\Delta$, and $\Sigma$, which reduces overall computation time and improves the performance of your code. **You will need to find the derivatives such as $\partial_r\alpha$ on your own.** You may find that first finding the derivatives of intermediate quantities such as $\Sigma$ and $\rho^2$, then using the chain rule can significantly simplify the process. To use the class above, you can use a code snippet such as the following:

```
boyer_lindquist_metric metric(a, M);

auto dydx = [&metric] (double x, const std::vector<double> &y) {
  metric.compute_metric(y[0], y[1]); // y[0] is r, y[1] is theta

  // compute the right hand sides of equations (20)-(25)
}
```

The code above defines a system of 6 coupled ordinary differential equations, which means that your `y` and `dydx` will both be a `std::vector<double>` of size 6. It is recommended that you use an adaptive Runge-Kutta solver from Homework 5 to solve this system. Since this project will be performance-sensitive, we will turn on all compiler optimizations to improve performance. Compile your code with the `-O3` option, which will significantly speed up the calculation. To further improve performance, take a look at the *bonus objective* at the very end of this document.

To validate your code, I will now describe two test cases that you can implement. Use these to convince yourself that your equations are written correctly and the code is solving them in the right way. Incorporate these test cases in your project report.

## 2.1 Einstein Ring in Schwarzschild Spacetime

Our first test concerns a non-rotating black hole ($a = 0$). Since the black hole can significantly bend the trajectory of light passing close to it, for an observer at distance $r_{\text{obs}}$ away from the black hole, there exists an angle $\xi$ where if the observer shoots a beam of light in that direction, the light will wrap around the black hole and come back to the observer. If the observer shoots light in all directions, they will see a ring of light that wraps through the black hole and comes back. This ring is called the *Einstein ring*. Depending on how many times the light wraps around the black hole, you might see a series of (fainter and fainter) photon rings.

The critical angles $\xi$ were calculated by Müller (2008). I'm quoting some results here that you can directly use:

- $r_{\text{obs}} = 10M$, $\xi = 0.48857874$. The light goes around the black hole and comes back to the observer.
- $r_{\text{obs}} = 20M$, $\xi = 0.24904964$. The light wraps around the black hole once, and then comes back to the observer, traveling a total $\phi = 4\pi$.

To construct initial conditions from these, you can set the initial position of the photon to be on the equatorial plane, $\theta = \pi/2$ and $\phi = 0$. The initial momentum is determined by $\xi$:

$$u_r = -\sqrt{g_{rr}}\cos\xi, \quad u_\theta = 0, \quad u_\phi = \sqrt{g_{\phi\phi}}\sin\xi \tag{26}$$

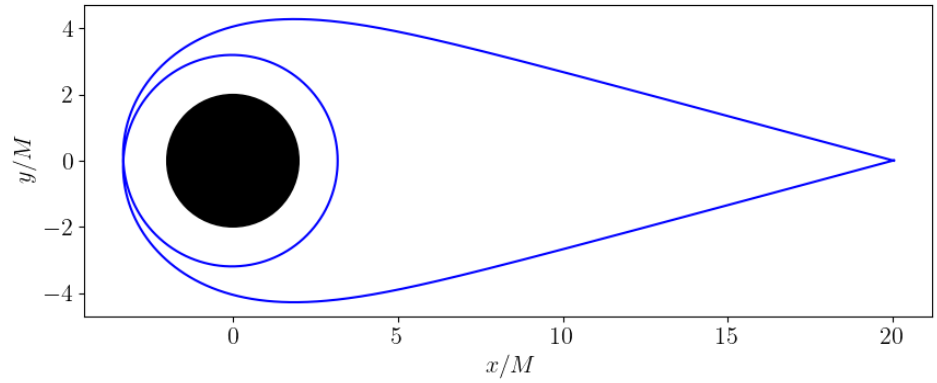



Figure 1: Example of the photon trajectory in Schwarzschild spacetime that comes back to the original position. This is the second test case in Section 2.1. The black circle is the event horizon.

**Your task:** Plot the trajectory of the photon for these two cases and confirm that the light rays come back to the original position. An example output will look like Figure 1. Since we are using an adaptive method, there is no guarantee that the integration will exactly go through the original point. Therefore, you will need to use a combination of *dense output* and linear interpolation to force the output to contain $\phi = 2\pi$ (going out and come back) or $\phi = 4\pi$ (circle around the black hole once and come back), and see how much you missed the starting $r_{\rm obs}$. This gives you an idea of the error in the integration. Discuss this error in the project report.

## 2.2 Unstable Spherical Photon Orbits in Kerr Spacetime

Teo (2003) discovered a series of spherical orbits for photons surrounding a maximally spinning black hole ($a = 1$). These spherical orbits are unstable, so a numerical scheme typically can follow their trajectories for some time but eventually the photon will escape from the starting radius and either fly to infinity or fall into the black hole. These serve as excellent numerical tests since the higher your precision is, the longer you can keep the error low and the photon in the desired orbit. The initial conditions for these unstable orbits are listed in Table 1.

**Your task:** Implement and run the unstable orbits. Plot the trajectories you get from these orbits until they go unstable. Plot the fractional error on the radius as a function of time. The plots may look like Figure 2. You can create 3D plots in Python using either matplotlib, or a modern package called K3D. You should plot the error evolution of different orbits on the same graph to compare how quickly they become unstable. Discuss how well your algorithm is doing.

| Orbit | $r$ | $u_\theta$ | $u_\phi$ |
|---|---|---|---|
| A | $1+\sqrt{2}$ | $\sqrt{11+8\sqrt{2}}$ | $0$ |
| B | $1+\sqrt{3}$ | $\sqrt{12+8\sqrt{3}}$ | $-1$ |
| C | $3$ | $\sqrt{27}$ | $-2$ |
| D | $1+2\sqrt{2}$ | $\sqrt{-13+16\sqrt{2}}$ | $-6$ |
| E | $2$ | $\sqrt{16}$ | $1$ |

Table 1: Initial conditions for unstable spherical orbits. All of them start on the equatorial plane with $\theta = \pi/2$ and $\phi = 0$. Since these are spherical orbits, the initial photon momentum in the $r$ direction $u_r = 0$, and should remain so.

When plotting the trajectory, you should use the pseudo-Cartesian coordinates:

$$x = \sqrt{r^2 + a^2}\sin\theta\cos\phi, \quad y = \sqrt{r^2 + a^2}\sin\theta\sin\phi, \quad z = r\cos\theta. \tag{27}$$

This is because the limit of $M \to 0$ of the Boyer-Lindquist coordinates is the oblate spheroidal coordinates which is related to the Cartesian coordinates by equations (27). In the limit of $a \to 0$, this is just spherical coordinates.

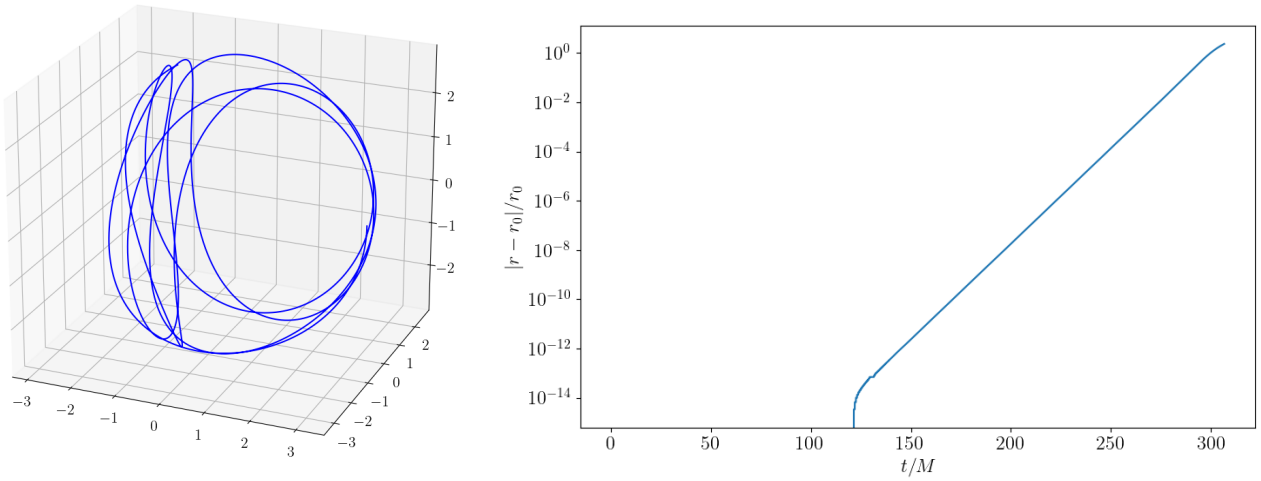



Figure 2: Example unstable photon orbit plots for Orbit C described in Section 2.2. The error starts to increase exponentially around $t \sim 125$ and the photon starts to significantly deviate from its intended trajectory at around $t \sim 280$.

# 3 Creating a Black Hole Image

Now that you are satisfied with the correctness of your numerical integration code, we shall use it to create an image of the black hole. Since by definition, light cannot escape from within a black hole, what we are really creating is an image of the bright disk surrounding the black hole. This disk consists of material attracted by the black hole and falling into its gravitational potential. As the material falls, its angular momentum prevents it from falling directly into the event horizon. Instead, it orbits the black hole and forms an *accretion disk*. The material in the disk slowly loses angular momentum due to friction in the disk, so the whole disk slowly falls closer and closer to the black hole. During this process, the enormous gravitational energy is converted to kinetic energy of the material, heating it to very high temperatures that can go up to $10^9\,\mathrm{K}$ for some systems. This is why the accretion disk can be very bright. The study of black hole accretion is still an active research area!

For our project, we will approximate the accretion disk as an infinitely thin ring sitting at $\theta = \pi/2$, and spanning some range of radii. In reality, although accretion disks can extend to very large distances, only the innermost part is the hottest and brightest, so truncating our thin ring at a finite radius makes sense. As an example, you can try to put the disk between $r_{\mathrm{in}} = 5M$ and $r_{\mathrm{out}} = 20M$.

To raytrace an image, we are going to start with each pixel on a viewing screen and trace the photon backwards. The light ray will do one of 3 things (i.e. 3 stopping conditions):

- Fall into the black hole
- Escape to infinity (some large distance)
- Hit the accretion disk

In the first two cases, you can assign the brightness of the pixel to 0. In the third case, you can then use the location where the light ray hits the accretion disk to calculate a brightness. For testing, you can simply set it to 1.

There are several ways to check whether the photon hits the accretion disk. One way is to check whether the trajectory gets within $r_{\mathrm{in}} \leq r \leq r_{\mathrm{out}}$ and $|\theta - \pi/2| < \epsilon$ for some tolerance $\epsilon$. The other, possibly more accurate way is to keep a `y_prev` in your stop condition, and check when $\theta$ changes sign while the radius is in the range of the disk. For the "falling into black hole" condition, since the photon will basically be stuck once it hits the horizon and spend an infinite amount of time there, it's prudent to terminate the ODE solver once you hit for example $r \leq 1.01 r_H$.

In order to make use of the photon orbit integrator you constructed earlier, you will need the initial conditions for every photon ray you shoot from the pixel on the viewing screen. Suppose the observer is located at a polar angle $\theta_0$ and distance $D$ from the center of the black hole, where $D \gg r_{\mathrm{out}}$ is very far away from the outer extent of the disk. It is customary to use the "apparent impact parameters" $\alpha$ and $\beta$ to describe the position of the ray on the observer's screen. They are related to the conserved quantities (angular momentum, energy, and Carter's constant), so we can use these impact parameters to compute the initial photon momenta (see Cunningham & Bardeen (1973) for reference):

$$\frac{u_\phi}{u_t} \approx -\alpha \sin\theta_0, \quad \frac{u_\theta}{u_t} \approx -\beta \tag{28}$$

Note that the negative sign on both of these values are inserted because we are tracing the photon *backwards* towards the black hole, instead of having the light coming towards us, which was the original convention.

Now that we have $u_\theta$ and $u_\phi$ we still need $u_r$ to complete our initial momenta. This can be done by solving the constraint that $g^{\mu\nu}u_\mu u_\nu = 0$. Specifically, if we write out all components in this equation:

$$g^{tt}u_t^2 + 2g^{t\phi}u_t u_\phi + g^{rr}u_r^2 + g^{\theta\theta}u_\theta^2 + g^{\phi\phi}u_\phi^2 = 0, \tag{29}$$

it can be solved by simply isolating the $u_r$ term:

$$\frac{u_r}{u_t} = -\sqrt{-\frac{1}{g^{rr}}\left(g^{tt} + 2g^{t\phi}\frac{u_\phi}{u_t} + g^{\theta\theta}\left(\frac{u_\theta}{u_t}\right)^2 + g^{\phi\phi}\left(\frac{u_\phi}{u_t}\right)^2\right)}. \tag{30}$$

The overall minus sign in front of the square root is again to make sure that the photon is going backwards from the screen to the black hole. Finally, since we don't really care about the photon energy but only its direction, we can set $u_t = 1$, which fixes the initial momentum of every photon given its screen position $(\alpha, \beta)$. As for initial location of the photons, it is less crucial, and you can start all of them at exactly the same observer point $r = D$, $\theta = \theta_0$, and $\phi = 0$.

While you are debugging your code, start modest and use a small image. Once you see everything is working, you can increase the image resolution and leave the computer to work. Figure 3 shows what a low-res image and a hi-res image might look like. The low-res image keeps most of the salient features, so you should be able to at least know that your code is working correctly. For an explanation of the image you obtained from the ray tracing calculation, the NASA official visualization gives you an extremely good breakdown of which part corresponds to what (Figure 4).

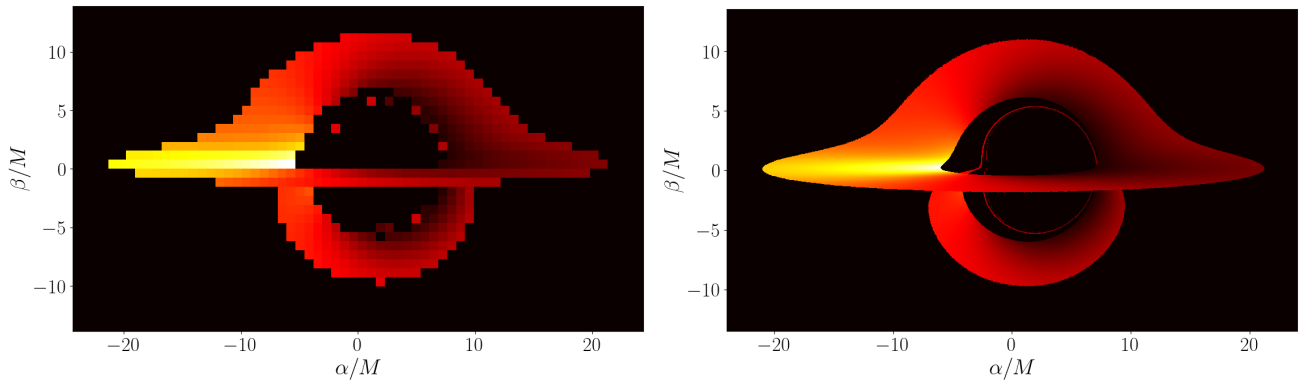



Figure 3: Example raytraced images of a black hole with $a = 0.99$. Left: $64 \times 36$ resolution. Right: $1920 \times 1080$ resolution. These are produced with observer distance $D = 500$, inclination $\theta_0 = 85°$, accretion disk radii $r_{\rm in} = 5M$ and $r_{\rm out} = 20M$. These images include gravitational redshift and Doppler boosting. The color scale simply show brightness, and should not be understood literally to be the color of the accretion disk. The calculation for the low-res image took about 0.2 seconds on my 16-core laptop, while the high-res one took about 2 minutes.

Finally, we need to account for the effect of gravitational redshift and Doppler boost due to the rotation of the disk. Gravitational redshift changes the light frequency and intensity due to gravity, while Doppler boost is due to the rotation of the accretion disk. The part of the disk rotating towards us looks brighter and bluer, while the part that moves away from us looks dimmer and redder. The Doppler effect makes the disk looks brighter on one side and dimmer on the other.

We define the total redshift factor as the ratio between the emitted frequency and the observed frequency (note that it's $u_t$ not $u^t$):

$$1 + z = \frac{\nu_{\rm em}}{\nu_{\rm obs}} = \left(1.0 + \Omega \frac{u_\phi}{u_t}\right) / (-g_{tt} - \Omega^2 g_{\phi\phi} - 2\Omega g_{t\phi})^{1/2}, \tag{31}$$

where $\Omega$ is the rotating angular velocity of the disk. This expression includes both gravitational redshift and Doppler boosting. If you want to know more about how this is derived, refer to the classic paper by Luminet (1979), who drew the first black hole image *by hand* from these analytic calculations. We are interested in the intensity $I$ of light that we observe. The combination $I/\nu^3$ is a Lorentz invariant, so we can write:

$$I_{\rm obs} = I_{\rm em} \frac{\nu_{\rm obs}^3}{\nu_{\rm em}^3} = I_{\rm em} \frac{1}{(1+z)^3}. \tag{32}$$

For our project, let's assume for simplicity that the emitted intensity $I_{\rm em}$ is constant across the disk, and independent of frequency. Then the brightness of the pixel will simply be given by $1/(1+z)^3$. For the rotation frequency $\Omega$, let's assume the disk is rotating at the Keplerian angular velocity (see e.g. Novikov & Thorne, 1973):

$$\Omega = \frac{1}{a + r^{3/2}/\sqrt{M}}. \tag{33}$$

Note that when using equation (31), you can use the final photon position and $u_\phi$ on the disk to compute the redshift factor $1 + z$. However, remember that we are tracking photons *backwards*. Therefore, you will need to flip the sign of $u_r$, $u_\theta$, and $u_\phi$ for the purpose of computing $1 + z$, since the equation assumes that the photon originates from that point on the disk instead.

**Your task:** Ray trace a few black hole images of various inclination angles $\theta_0$ and spin parameters $a$. Feel free to choose your other parameters such as disk radii $r_{\rm in}$ and $r_{\rm out}$, distance to the observer $D$, and image resolution. Use a resolution that is reasonable for the amount of computational resources you have access to. As long as the images shows all the right features, then you will get all the points for this part.

In addition to making a few different images, please attempt to address the following in your experimentation:

- What is the main difference between the photon ring of a rotating $(a = 1)$ and a non-rotating $(a = 0)$ black hole?
- When you set $r_{\rm in} \approx r_H$, the inner edge of the disk essentially trace out the shape of the event horizon. Set the entire disk to be the same brightness so that you can see the inner edge

clearly. What does the event horizon look like in the rotating and non-rotating case? Note that since one of the stopping condition is hitting slightly outside the horizon, you probably don't want to take $r_{\rm in}$ to be exactly on top of the horizon.

- Replace the intensity in Equation (32) with simply the $\phi$ angle of the source point (the point where the light ray crosses the disk). Perform raytracing for a relatively small observer angle $\theta_0$ and a small $r_{\rm in}$, for a range of black hole spins $a$. Describe what the plots show and what is the trend.

Write up a report of what you did, including plots representing your work. Your report should include a basic description of your understanding of the problem, a description of the methods you chose and the rationale, and it should address all the "Your task" sections listed in the previous sections. The grade of this project is determined by whether you successfully completed the tasks, and by the quality of the report. Include the report as a pdf file in the repository, and submit all of your code (including C++ source files, headers, and the python files you use to make plots). Refer to your code in the report and explain briefly in the "methods" section which file does what, e.g. "The `xxx.cpp` file tests the integrator with unstable photon orbits." Provide enough information so that the grader can reproduce your results if needed. **Keep the report within 6 pages.** Extra pages will not be counted towards the project score.

**Bonus objective:** This part is worth 10 bonus points for undergraduate students taking Physics 4027, but mandatory for graduate students taking Physics 5027. The goal is to implement parallelization for the ODE solver to speed up the raytracing calculation.

Since the light ray launched from each pixel is completely independent from each other, this problem is "embarrassingly parallel" from the get go. We will use `OpenMP` to carry out the parallelization, which involves including the header file `omp.h`, and adding the following `pragma` line immediately above your loop that goes over the pixels:

```
#pragma omp parallel for
for (int i = 0; i < Ny; i++) {
  // ...
}
```

You will also need to add `-fopenmp` to the command you use to compile the source code, e.g.:

```
g++ -O3 -fopenmp bh_image.cpp
```

One complication however, is that only one thread can use an instance of the ODE solver object at a time. For example, if two different threads are using the same ODE solver object, then they would overwrite each other when both threads tries to push $y$ values to the `result` vector. Additionally, our `boyer_lindquist_metric` object contains the values of all the metric derivatives at a given position. If multiple threads tries to use the same object at the same time, they will get the metric values all messed up.

The solution to this complication is to define an `std::vector` of both the ODE solver (e.g. `rk45_dormand_prince`) and the `boyer_lindquist_metric` objects. Then, you can resize both vectors to the maximum number of threads that OpenMP is going to use on your system. This number

can be obtained using the global function `omp_get_max_threads()`. Then, in the parallel `for` loop, you can obtain the current thread id using the global function `omp_get_thread_num()`, and specifically use the ODE solver and metric object corresponding to that thread id in the loop to trace the particular ray. Finally, since different threads should write to different parts of the output array, there should be no conflict, so you shouldn't need to do anything special regarding the output.

I suggest tackling this part only when you are absolutely confident that your single-threaded version of the raytracer works correctly, at least at low resolutions. You do not want to bring in the complication of parallelization while trying to debug the basic algorithm. The goal of this part is simply to accelerate the calculation when you trace a big image.

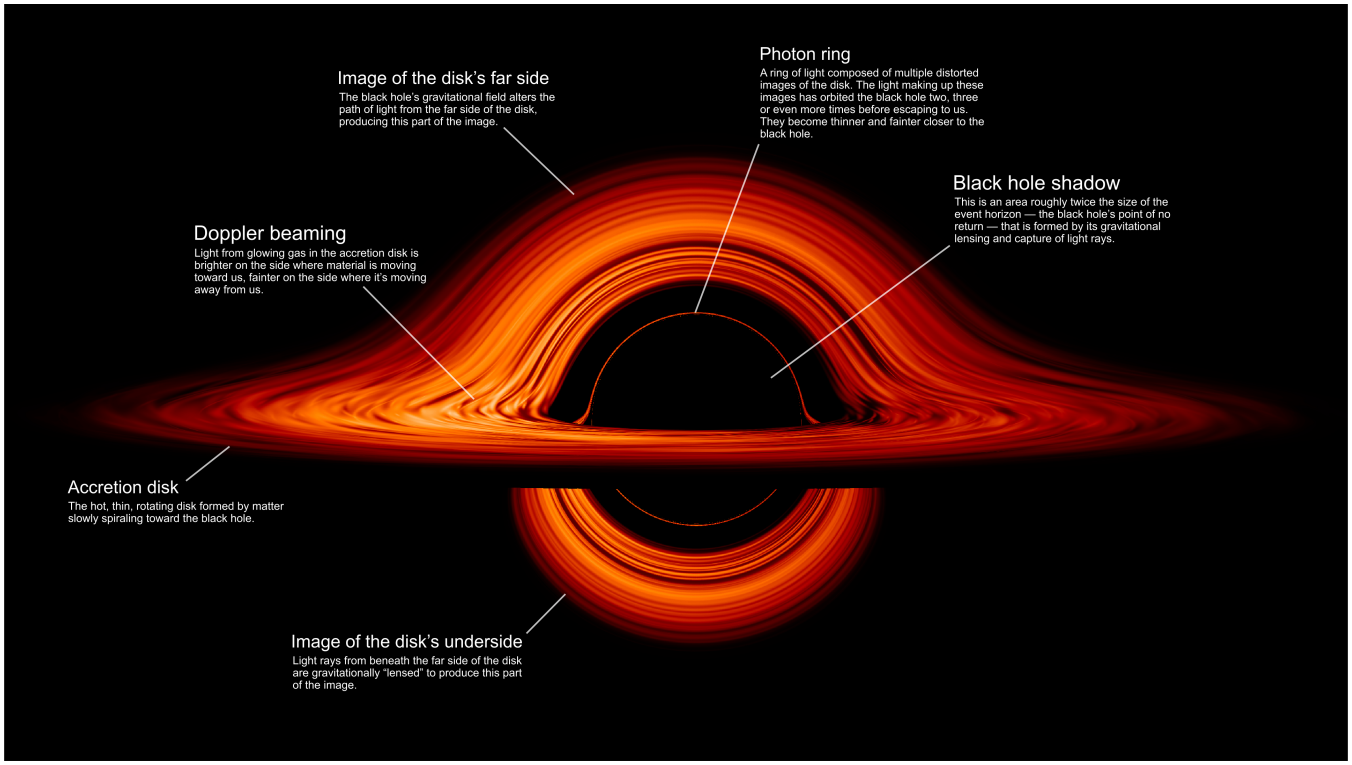



Figure 4: Explanation of various aspects of the black hole visualization by NASA. Their accretion disk has a lot of ripples in it, since they included some noise in the emitted intensity $I_{\rm em}$. If you want to reproduce such ripples, feel free to tweak your $I_{\rm em}$ to include such variations. They also only did a Schwarzschild black hole, which is why the photon ring looks so symmetric.

# References

Cunningham, C. T. & Bardeen, J. M. 1973, , 183, 237. doi: doi:10.1086/152223

Komissarov, S. S. 2004, Monthly Notices of the Royal Astronomical Society, 350, 427, doi: 10.1111/j.1365-2966.2004.07598.x

Luminet, J. P. 1979, Astronomy and Astrophysics, 75, 228

MacDonald, D., & Thorne, K. S. 1982, Monthly Notices of the Royal Astronomical Society, 198, 345, doi: 10.1093/mnras/198.2.345

Müller, T. 2008, Physical Review D, 77, 124042, doi: 10.1103/PhysRevD.77.124042

Novikov, I. D., & Thorne, K. S. 1973, Astrophysics of black holes, in Black Holes (Les Astres Occlus), 343–450

Teo, E. 2003, General Relativity and Gravitation, 35, 1909, doi: 10.1023/A:1026286607562